

System Design Architecture Documentation using AI



So ideation of the project is done, then features, and we filtered our features too, and then we created our PRD.

So till now, whatever we have done is the work of a Project Manager.

So now you can think of the PRD as being transferred to the Engineering Team.

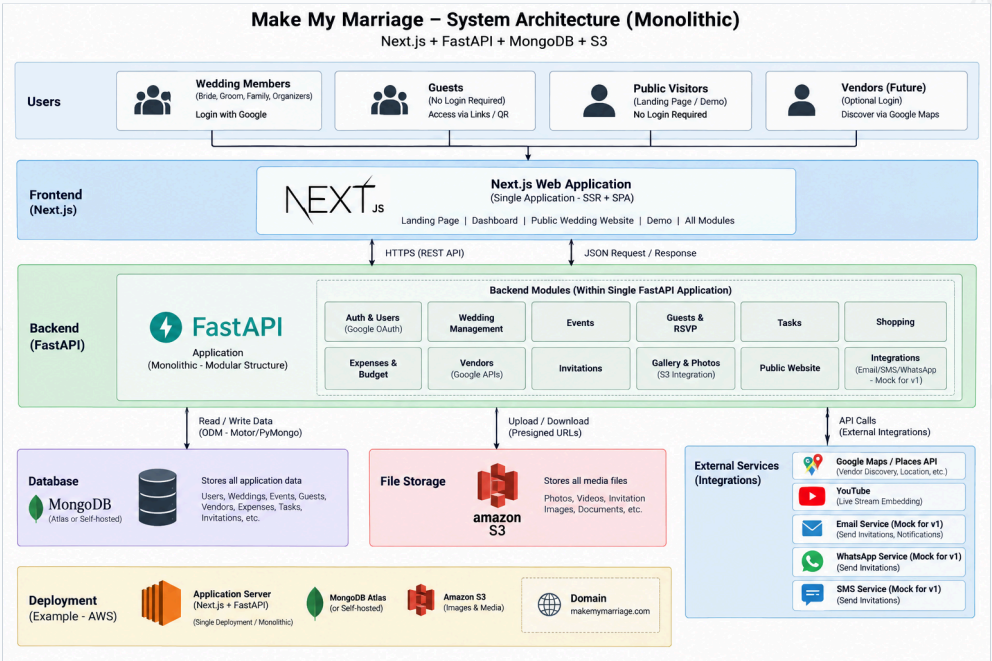
Now you are the lead or developer of the project. You need to decide the architecture, tech stack, but we will just supervise as we did before, and we will use the API only.

So I will give a prompt and provide my preferences.

“PRD looks good to me. Let’s develop a system design architecture diagram for me, and let me provide our preferences. I want a monolithic app. Use Next.js for frontend, for DB use MongoDB, and in backend we will use FastAPI + MongoDB. For image storage, we will use S3. But suppose if you are confused, you can ask AI to ask you questions so that the tech stack and tools can be finalized, and it will decide based on what it decides.”

You can discuss how things will happen, how it will keep the files, where the backend will reside, and ask ChatGPT to make you understand. It’s unfair to continue blindly somehow, because most of the time it will correct, but if you control the things, it would be great for you.

So I have created a diagram. See, this came in one go. We can change it after another prompt, and it might be that in the future we will change this too, so use your way.



We will use MongoDB Atlas database, and use its free tier for some data.

Read this blog to set up MongoDB. [Create MongoDB Database](#)

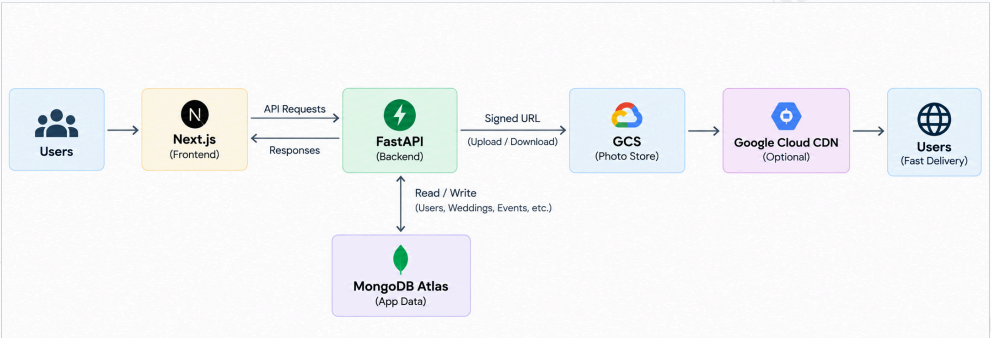
As of now, we decided to use **Mongoose** to query in DB. If anything might change, we will discuss later.

Okay, the DB we are using is just for data, not for physical image files. You can store, as our code looks first storing on GitHub and Google Drive, same way MongoDB for URLs of images, not for exact image data.

For that, S3 or GCS or CloudFront can be used.

Your responsibility is to be curious and keep chatting with your ChatGPT or whatever AI assistant you are using to make decisions.

So for images, currently I have thought to use Google services. We will see later. For image processing, we will rely on Next.js first. We will see later, or we might use a CDN.



For email, we will use **Resend APIs**.

There are many other providers also you can use. Resend gives 100 emails/day free. We will not send so much mail, else we need to use email providers.

Because if we need to send mail to 1000 people at once, we need to properly design the architecture, because sending and calling 1000 APIs at once is not a good way.

So we will not complicate email. We will just send at once. We will not do all at once, or will use some alternative way.

Now see, this is why you can brainstorm: what is the right way? Is it good to send 1000 emails at once? If not, how does mass email work? So we need to be curious about how things happen.

We are in the era of AI. Ask everything that comes to your mind. So decide in that way.

Here, whatever we are doing or whatever we will do next, doesn't mean all will be the same for you as well. So nothing is perfect, but try to be perfect and understand. Don't go blindly, and never be demotivated if things don't come as planned here or will come next, okay?

AI is there, don't worry. 😊

So, what I mean to say is, over every feature you listed or thought of should be properly planned, and each answer should have **who and what** will you use to make that possible.

That's what we are doing above, and you will do the same.

Like, as we discussed, any guest can upload an image, but can the upload be done on a public URL?

The answer is no. This way, they can spam our database.

So we will have a **token-based, expiring URL** that will expire after some time. Within that time frame, the shared person can upload to the URL.

Okay, note this also.

For deployment, we will use **Vercel**. You can use any other provider for yourself.

There are EC2 instances also. An Amazon EC2 instance is a virtual server in the Amazon Web Services (AWS) cloud that lets you run applications on demand.

You can use Netlify or anything; it's your choice. But later, we will also learn by moving things from Vercel to AWS. So we will learn migration also.

So again, I'm repeating: you can ask your LLM to ask questions to you that you can answer.

So please, if required, do ask ChatGPT to ask questions to you to design the product properly.

So here is the summary of my architectural summary of what we will do.

Summary of our Design

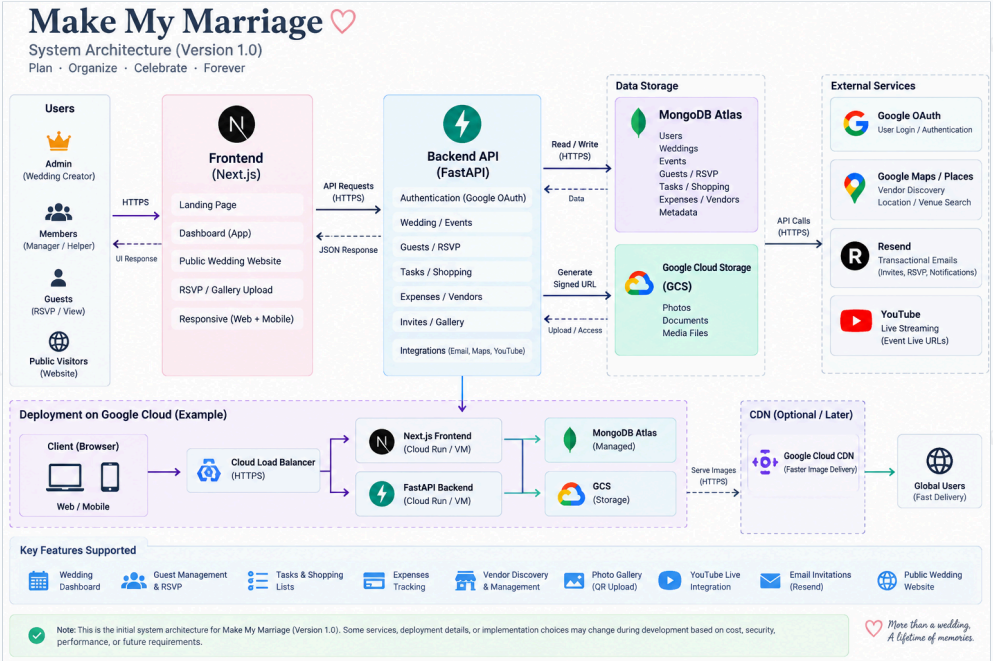
For **Make My Marriage**, we will build a **modular monolithic system** using **Next.js for the frontend**, **FastAPI with Python for the backend/API**, and **MongoDB Atlas for application data**. The wedding will be the main entity, where **1 account creates 1 wedding**, and the Admin can invite family members/organizers such as Rahul as **Manager or Member** through an email invitation link; they will sign up/login using **Google OAuth**, while guests will not need an account. The dashboard will provide an overall view of the wedding, including countdown, events, guests, RSVP, tasks, shopping, expenses, vendors and other activities. The Events module will support default events like **Haldi, Mehndi, Sangeet, Wedding and Reception**, along with custom events, and each event can have its own guests, tasks, expenses, vendors, photos and YouTube Live link. Guest management will use one central guest collection with event-wise attendance/RSVP mapping, while Tasks will manage wedding activities and Shopping will separately manage items to buy and their

status. Expenses will track the overall wedding budget, event-wise expenses, payments, vendors and receipts. Vendor functionality will have **Vendor Discovery using Google Maps/Places APIs** and Wedding Vendor Management where selected vendors can be attached to events and their quotes, payments, documents and details can be managed. Invitations will support pre-built digital invitation designs with shareable links/QRs and **Email, WhatsApp and SMS will initially be mocked**, while actual email sending can use **Resend**. Every wedding will automatically have a **public wedding website** generated from its wedding data, with couple details, countdown, events, venue, RSVP, gallery and live-stream information. Photos and documents will be stored in **Google Cloud Storage (GCS)** rather than MongoDB; FastAPI will generate **signed/presigned URLs** so users and guests can upload photos directly to GCS, and **Google Cloud CDN can be added later** for faster gallery delivery at scale. The Gallery will support event-based albums, private galleries by default, shareable gallery links and **QR-based guest photo uploads without login**. YouTube Live URLs will be stored per event and displayed on the public wedding website. Documents can store vendor contracts, receipts, bills and booking documents. The system will also have a public landing page and a fully populated **Ashu's Marriage demo wedding** so visitors can explore the product. The main flow will be **Next.js → FastAPI → MongoDB Atlas**, while media will use **FastAPI → signed URL → GCS**, and external integrations will include **Google OAuth, Google Maps/Places, YouTube and Resend**. For deployment, our current assumption is to deploy the Next.js and FastAPI application as a **single monolithic application environment**, with MongoDB Atlas as the managed database and GCS for media storage; the exact cloud deployment can be finalized during implementation. We will keep clear module boundaries inside the monolith so that individual modules can be separated later if the product grows. **This is our current system-design baseline; some libraries, providers, deployment details or implementation choices may change during development based on cost, security, performance, or practical requirements.**

Again, I'm saying this is how I have designed it, because from videos, it can't be possible to do it exactly. So let's try our way.

You can try your way. That's why I repeat: focus on the idea and not just copy.

Note: More time you spend and discuss with AI, less time you debug your project.



Download System Design Architecture PDF and digram Make My Marriage System Design Okay, we'll discuss more in the next chapter.

See you, bye-bye! ☐



LEARN BY VIDEO

Explore courses on NamasteDev.com

Hands-on video lessons from Akshay Saini's ecosystem.

Start learning ↗